

2026 VLSI Design and Implementation

LAB5 Error Detection

Objective

In this lab, students will design a 3x3 adjugate matrix calculator and study the methods for detecting and recovering the circuit error from runtime setup violations.

Due Date: 2026/6/19 12:00 p.m. (3 weeks)

Lab5 Download

1. Download the provided environment using the following command.
tar xvf /home/TA00/VLSI_2026/Lab5.tar
2. The extracted LAB directory contains:
 - **00_TESTBED**: Contains the testbench and pattern files.
 - **01_RTL**: Your workspace for the Verilog design (**AMED.v**).
 - **02_SYN**: Synthesis scripts and working directory.
 - **03_GATE**: Gate-level netlist and simulation files.

Block Diagram

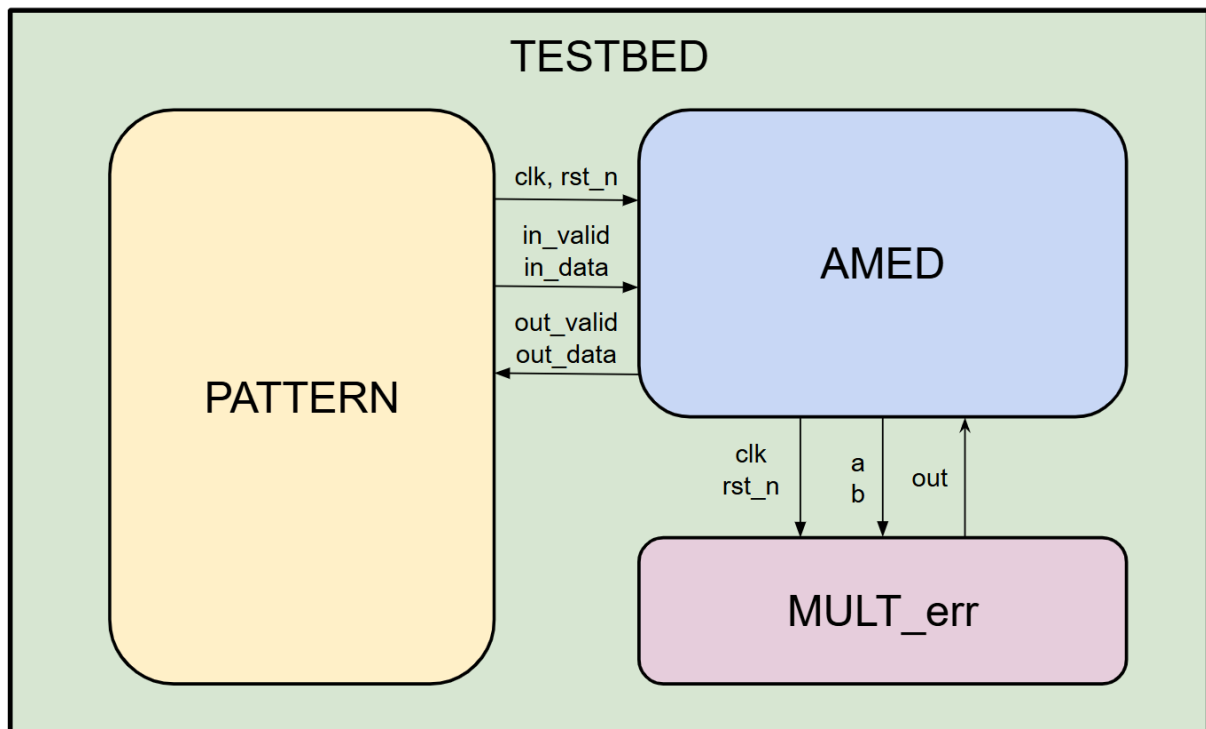


Fig. 1. Design Block Diagram

Design Description

In this lab, instead of calculating the exact inverse matrix (which requires complex floating-point division), your hardware will compute the **Adjugate Matrix** of a 3x3 input matrix.

1. Adjugate Matrix

For a given 3x3 matrix **A**, its adjugate matrix **Adj(A)** is the **transpose of its cofactor matrix**. Each element in the adjugate matrix is calculated by finding the determinant of a 2x2 submatrix.



$$\text{adj } A = \begin{bmatrix} + \begin{vmatrix} a_{22} & a_{23} \\ a_{32} & a_{33} \end{vmatrix} & - \begin{vmatrix} a_{12} & a_{13} \\ a_{32} & a_{33} \end{vmatrix} & + \begin{vmatrix} a_{12} & a_{13} \\ a_{22} & a_{23} \end{vmatrix} \\ - \begin{vmatrix} a_{21} & a_{23} \\ a_{31} & a_{33} \end{vmatrix} & + \begin{vmatrix} a_{11} & a_{13} \\ a_{31} & a_{33} \end{vmatrix} & - \begin{vmatrix} a_{11} & a_{13} \\ a_{21} & a_{23} \end{vmatrix} \\ + \begin{vmatrix} a_{21} & a_{22} \\ a_{31} & a_{32} \end{vmatrix} & - \begin{vmatrix} a_{11} & a_{12} \\ a_{31} & a_{32} \end{vmatrix} & + \begin{vmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{vmatrix} \end{bmatrix}$$

Fig. 2. 3x3 Adjugate Matrix

Module description:

- Input:** Nine **8-bit signed integers** (72 bits of **A_11** to **A_33**).
- Output:** Nine **24-bit signed integers** (216 bits of **A_11** to **A_33**).
Note: The bit-width is expanded to 24-bit to prevent overflow during multiplication and subtraction.

2. Multiplier IP with error

To simulate a real-world environment, we provide a multiplier IP (MULT_err). You **MUST** use this IP for all multiplications in your design. Direct usage of the “*” operator for multiplication is **strictly prohibited**.

Usage Rule

The output of **MULT_err** **CANNOT** be directly connected to other combinational logic (such as adders or multiplexers).

- You must connect the output of the IP directly to a Flip-Flop (Register) immediately. All subsequent calculations (e.g., subtracting two

multiplied terms) must be performed using the values stored in these registers.

- Failing to store the IP output will cause the temporary error signals (glitches) to propagate and corrupt your entire data path.

Error Mechanism & Timing Behavior

This IP intentionally simulates a **Setup Time Violation**. There is a certain probability (e.g., 5%) that the multiplication will take longer than a standard clock cycle. You must fully understand its behavior across different clock phases:

Scenario A: When an Error Occurs (Setup Violation)

- At **posedge clk**: The computation is not yet finished. The IP outputs unstable/corrupted data (garbage values).
- At **negedge clk (After Half Cycle)**: The computation finally completes. The IP outputs the correct data.

Scenario B: When NO Error Occurs (Normal Operation)

- The computation finishes early. The IP outputs the correct data at **both posedge clk and negedge clk**.

How to Detect the Error

To catch these random setup violations, you should implement a dual-latching mechanism. Since the inputs are held stable, you can rely on the timing difference between the two clock edges:

1. **Main Register**: Triggered at **posedge clk**. It captures data to proceed with the main data path.
 - *If an error occurs*: It captures the corrupted data.
 - *If no error occurs*: It captures the correct data.
2. **Shadow Register**: Triggered at **negedge clk** (half a cycle later).
 - Because the data is guaranteed to stabilize by the falling edge (regardless of whether an error occurred), the Shadow Register will **always capture the correct data**.
3. **Error Checker**: Compare the values of the Main Register and the Shadow Register.
 - If they do not match, an error is detected. You must recover the correct data from the Shadow Register to Main Register.

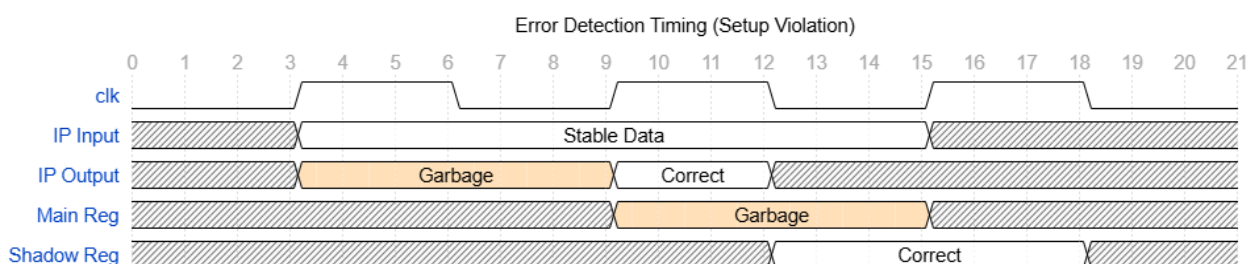


Fig. 3. Example Waveform of IP with Error

Signal Description

Signal	In / Out	Bit Width	Description
clk	In	1	System Clock.
rst_n	In	1	Asynchronous active-low reset.
in_valid	In	1	High when the input in_data is valid.
in_data	In	72	The input 3x3 matrix. Consists of nine 8-bit signed integers. ※Format: From A_11 is at [71:64], A_12 is at [63:56], ..., A_33 is at [7:0].
out_valid	Out	1	High when the output out_data is valid.
out_data	Out	216	The computed 3x3 adjugate matrix. Consists of nine 24-bit signed integers. ※Format: From A_11 is at [215:192], ..., A_33 is at [23:0].

Table. 1. AMED Module I/O Port List

Signal	In / Out	Bit Width	Description
clk	In	1	System Clock.
rst_n	In	1	Asynchronous active-low reset.
a	In	16	16-bit signed multiplicand.
b	In	16	16-bit signed multiplier.
out	Out	32	32-bit signed multiplication result ($a * b$).

Table. 2. MULT_err Module I/O Port List

Design Constraints

01_RTL

1. Top module name: `AMED.v`
2. **All flip-flops in your design (including FSM, data path registers, and output registers) MUST be positive-edge triggered (posedge clk).**
The ONLY flip-flops allowed to use negative-edge triggering are the Shadow Registers used for error detection.
3. The reset signal (`rst_n`) would be given only once at the beginning of simulation. All output signals should be reset after the reset signal is asserted.
4. The `out_data` should be zero after your `out_valid` is pulled down.
5. The `out_valid` signal cannot overlap with the `in_valid` signal at any time.
6. The `out_valid` signal should be high in **one cycle**.
7. The next inputs will come in the next 2 ~ 5 negative edges of the clock after your `out_valid` is pulled down.
8. The execution latency is limited to 1000 cycles for each Matrix.
9. **Your design must pass all hidden testcase.**

Before running synthesis, please read the "02 Synthesis Flow" instructions in the final Note section

02_SYN

1. **You are NOT allowed to synthesize any multipliers other than the provided MULT_err IP.** Direct use of the * operator in your RTL code is strictly prohibited.
Note: You can verify your result by checking the AMED.resource or AMED.reference reports after running synthesis. If you find any standard synthesis multipliers (e.g., DesignWare components containing mult) other than your instantiated MULT_err, you have violated this rule and must revise your RTL.
2. The synthesis result cannot contain any **latch**.
3. The input / output delay is set to 0.5 * (clock period), and the output loading is set to 0.05.
4. You can adjust your clock period by yourself. The precision of the clock period is 0.1, for example, 45.0 is allowed, 44.55 is not allowed.
5. After synthesis, the design must meet timing requirements. The area report is valid only if the final slack reported in `AMED.timing` is **MET**.

03_GATE

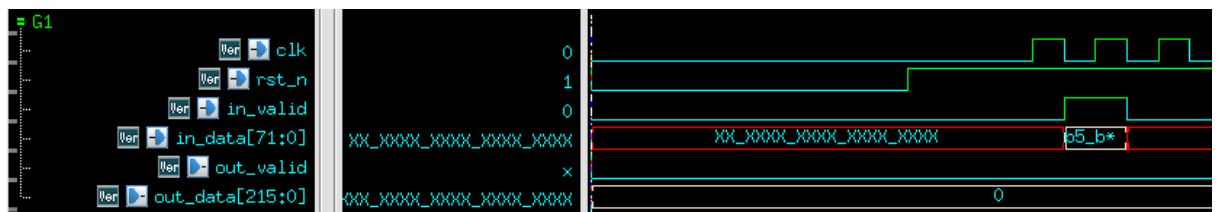
1. All requirements in 01_RTL must be met after including timing checks in Gate-Level simulation.

Waveform Examples

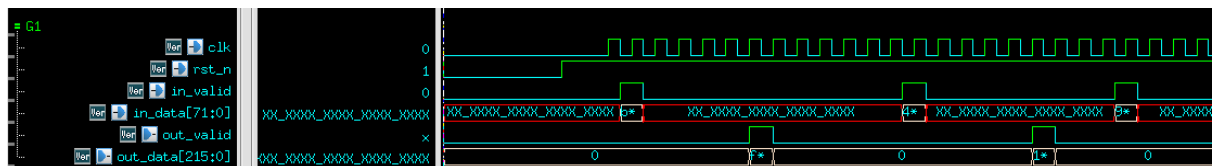
1. The reset signal would be given only once at the beginning of simulation.



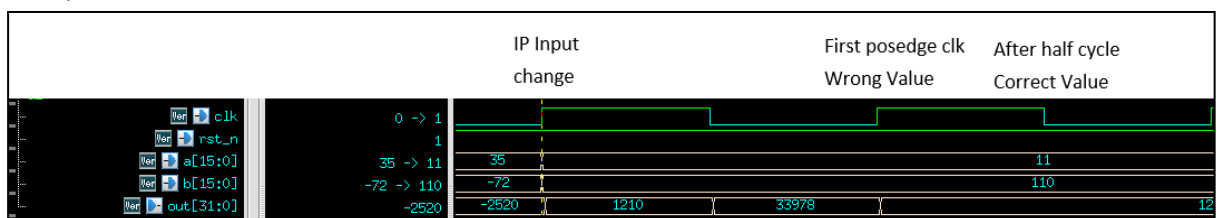
2. Input signals will be sent in one cycle.



3. The next input will come in the next 3 ~ 7 negative edges of the clock.



4. When the MULT_err IP encounters an error, the value sampled at the posedge of the clock will be wrong, while the value at the negedge (half cycle later) will be correct.



Grading Policy

1. **Function Validity: 70%**

Passing the simulation without error generate from MULT_err IP (**More detail in the final Note section**) - 30%

Passing the simulation with error generate from MULT_err IP - 40%

2. **Performance: 30%**

Performance Score = Power * Area * CYCLE_TIME * Latency

※Power is "Total Power" in your power report.

※Area is "Total cell area" in your area report.

※**Latency** is the clock cycles between the falling edge of the in_valid and the rising edge of the first out_valid.

※**CYCLE_TIME** is defined by yourself.

Note: You must pass all simulations to participate in the performance ranking.

3. **Due date: 2026/6/19 12:00 p.m.**

Please submit the following zip files to the E3 system before 12:00 at noon in Jun. 19

- vlsixx.zip (ex. vlsi00.zip)
 - vlsixx (folder)
 - AMED.v
 - {your cycle time}.txt (ex. 10.6.txt)

4. If uploaded files **violate the naming rule**, you will get **5 deduct points**.

Notes

- **Submission**

- 1st Demo

Submit and pass all test cases before 12:00 noon on Jun 19, 2026.

The score will be calculated according to the grading policy.

- 2nd Demo

If you do not pass all test cases or fail to submit (in the 1st Demo), and submit and pass all test cases before 12:00 noon on Jun 26, 2026, you will receive the Function score(70%).

- **02 Synthesis Flow**

To prevent Synopsys Design Compiler (DC) from erroring out on simulation-only syntax (random number in MULT_err_SIM.vp), we implement a **File Swapping & Removal Flow**.

During synthesis, the **MULT_err_SIM.vp** IP will be replaced by a clean, **standard multiplier (MULT_err_SYN.v)** to allow the tool to accurately calculate the cell area. Right before exporting the netlist, **this normal multiplier will be completely stripped out** to leave an empty wrapper for gate-level simulation.

The process is split into **TWO steps**:

1. **Step 1: ./01_run_dc_shell (Main Synthesis with Standard Multiplier)**

- This script reads **MULT_err_SYN.v** (a clean combinational multiplier view) instead of the simulation file.

- DC performs optimization, accurately maps the hardware multiplier area, and saves the database into a `.ddc` file.

2. Step 2: `./02_remove_mult` (Netlist Export & Multiplier Removal)

- This script restores the `.ddc` database and executes `remove_design MULT_err`, **unlinking and pulling the normal multiplier out of the netlist**.
- It then exports the final `AMED_SYN.v` netlist and `AMED_SYN.sdf`, leaving `MULT_err` as an empty black box instance.
- **Note:** "unresolved references" warnings during this step are expected. Please safely ignore them.

IMPORTANT: Your Gate-level simulation (`03_GATE`) requires the empty black box instances generated by Step 2 so that the simulation engine can re-link them back to the protected simulation IP (`MULT_err_SIM.vp`).

● Reference Commands

- 01_RTL/ (RTL simulation) `./01_run_vcs_rtl`
- 02_SYN/ (Synthesis) `./01_run_dc_shell`
`./02_remove_mult`
- 03_GATE / (Gate-level simulation) `./01_run_vcs_gate`

● MULT_err IP Usage

To ease your debugging process in the early stages, the `MULT_err` IP comes with flexible configurations:

1. Disabling the Error Injection:

Inside the `MULT_err_SIM.vp` file, you will find a macro definition at the top:

```
`define ENABLE_RANDOM_ERROR
```

If you comment out this line, the IP will function as a normal multiplier.

2. Adjusting the Error Probability:

The IP has a parameter `ERROR_PROB` (default is 5, meaning a 5% error rate).

You can overwrite this parameter in the `MULT_err_SIM.vp` file to test the design under different error rates:

```
parameter ERROR_PROB = 8'd5 // change 5 to any number
```



```

1  `define ENABLE_RANDOM_ERROR
2  module MULT_err #(
3      parameter ERROR_PROB = 8'd5
4  )(
5      input      clk,
6      input      rst_n,
7      input  signed [15:0] a,
8      input  signed [15:0] b,
9      output signed [31:0] out
10 );

```

Fig. 4. MULT_err_SIM.vp file

※**TA GRADING POLICY**: > During the final grading, the TA will use the original IP with **ENABLE_RANDOM_ERROR turned ON** and **ERROR_PROB** set to **5**.